# TYPES DE DONNÉES ET STRUCTURES

Les briques de construction de Python

TYPES PYTHON

| | |
|---|---|
| `int` | `float` |
| `str` | `bool` |

# TYPES NUMÉRIQUES

int et float, les nombres de Python

```python
age = 25                # int
pi = 3.14159            # float
grand = 10 ** 100       # int sans limite !

type(age)     # <class 'int'>
type(pi)      # <class 'float'>

# Conversion automatique
resultat = age + pi     # 28.14159 → float
int(3.7)                # 3 (troncature)
float(42)               # 42.0
```

INT · FLOAT

int | float

# CHAÎNES DE CARACTÈRES

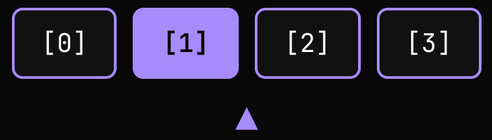
## str, le texte en Python

```python
nom = "Alice"
prenom = 'Bob'
phrase = "Bonjour le monde"

# f-strings : injection de variables
age = 30
msg = f"{nom} a {age} ans"
# "Alice a 30 ans"

# Méthodes courantes
"python".upper()        # "PYTHON"
"  salut  ".strip()     # "salut"
"a,b,c".split(",")      # ["a", "b", "c"]
len(phrase)             # 16
```

CHAÎNE INDEXÉE



# BOOLÉENS ET NONE

## Vrai, faux et l'absence de valeur

```
actif = True
type(actif)        # <class 'bool'>

# Comparaisons → booléens
5 > 3              # True
10 == 11           # False
"a" in "abc"       # True

# None : absence de valeur
resultat = None
type(resultat)     # <class 'NoneType'>
resultat is None   # True (pas ==)
```

TRUE VS FALSE

True **VS** False

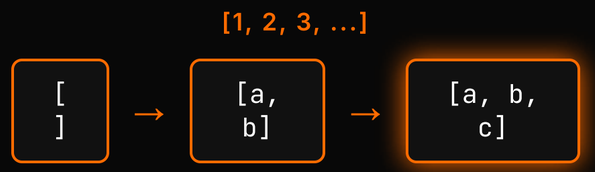
# LES LISTES

## La boîte à compartiments de Python

```python
fruits = ["pomme", "banane", "cerise"]
notes = [14, 17, 12, 19, 8]
mixte = [1, "hello", True, 3.14]

# Modifier sur place (mutable)
fruits.append("mangue")     # ajoute à la fin
fruits.remove("banane")     # retire par valeur
notes.sort()                # [8, 12, 14, 17, 19]

len(fruits)    # 3
"pomme" in fruits  # True
```



# INDEXATION ET SLICING

## Accéder aux éléments avec précision

```
lettres = ["a", "b", "c", "d", "e"]

# Indexation
lettres[0]       # "a" (premier)
lettres[-1]      # "e" (dernier)
lettres[-2]      # "d" (avant-dernier)

# Slicing [début:fin:pas]
lettres[1:3]     # ["b", "c"]
lettres[:3]      # ["a", "b", "c"]
lettres[2:]      # ["c", "d", "e"]
lettres[::2]     # ["a", "c", "e"]
```

LST[1:3]

# LES TUPLES

La boîte scellée de Python

```python
# Tuple : immuable
coordonnees = (48.8566, 2.3522)  # Paris
couleur_rgb = (255, 128, 0)
jours = ("lun", "mar", "mer", "jeu", "ven")

coordonnees[0]    # 48.8566
len(jours)        # 5

# Impossible de modifier !
# coordonnees[0] = 0  → TypeError

# Déballage (unpacking)
lat, lon = coordonnees
print(f"{lat}°N, {lon}°E")
```

MUTABLE VS IMMUTABLE

Mutable **vs** Immutable

# LES DICTIONNAIRES

## L'annuaire de Python

```python
etudiant = {
    "nom": "Alice",
    "age": 22,
    "notes": [14, 17, 12]
}

etudiant["nom"]           # "Alice"
etudiant["moyenne"] = 14.3  # ajouter
etudiant.get("ville", "?")  # "?" (défaut)

# Parcourir
for cle, val in etudiant.items():
    print(f"{cle} → {val}")
```

**CLÉ → VALEUR**

`"clé_1"` → valeur

`"clé_2"` → valeur

`"clé_3"` → valeur

# LES ENSEMBLES

## L'ensemble mathématique en Python

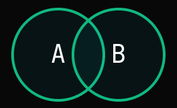
```python
fruits = {"pomme", "banane", "cerise"}
nombres = {1, 2, 3, 2, 1}  # → {1, 2, 3}

fruits.add("mangue")
fruits.add("pomme")   # ignoré (déjà présent)

# Éliminer les doublons
liste = [1, 2, 2, 3, 3, 3]
uniques = list(set(liste))  # [1, 2, 3]

# Opérations ensemblistes
a = {1, 2, 3}
b = {2, 3, 4}
```

UNIQUE ∩ U



# RÉCAPITULATIF

**Vos briques sont prêtes**

- ✓ int / float : nombres sans limite et décimaux
- ✓ str : texte immuable avec f-strings
- ✓ bool / None : interrupteurs et absence
- ✓ list : boîte à compartiments (mutable)
- ✓ tuple : boîte scellée (immuable)
- ✓ dict : annuaire clé-valeur
- ✓ set : ensemble sans doublons